Automating Data Import and Relationship Mapping Using Import Sets & Dot Walking

Problem Statement:

Your organization wants to migrate/export a list of new employees with their details, such as Name, Email, Department, and Manager, from an external system. Once imported, the system should be able to automatically show the manager's email of the employee's department.

Objective:

- Import employee records from an external CSV file into ServiceNow.
- Automatically map relationships between employees, departments, and managers using dot-walking.

Skills:

Import Sets, Dot Walking

Description:

Importing data from a Data Source in ServiceNow means bringing external data (like CSV, Excel, or database records) into the ServiceNow platform so it can be stored, processed, and used within tables such as Incident, User, or any custom table.

In ServiceNow, a Data Source defines where the data is coming from (for example, a CSV file or external database) and how it should be read. Once configured, the system loads the data into an Import Set table, which acts as a temporary staging area. From there, a Transform Map is used to map fields from the Import Set table to the target table. This ensures data is correctly inserted or updated based on defined field mappings and business rules.

• This process helps organizations:

• Avoid manual data entry

• Maintain data accuracy and consistency

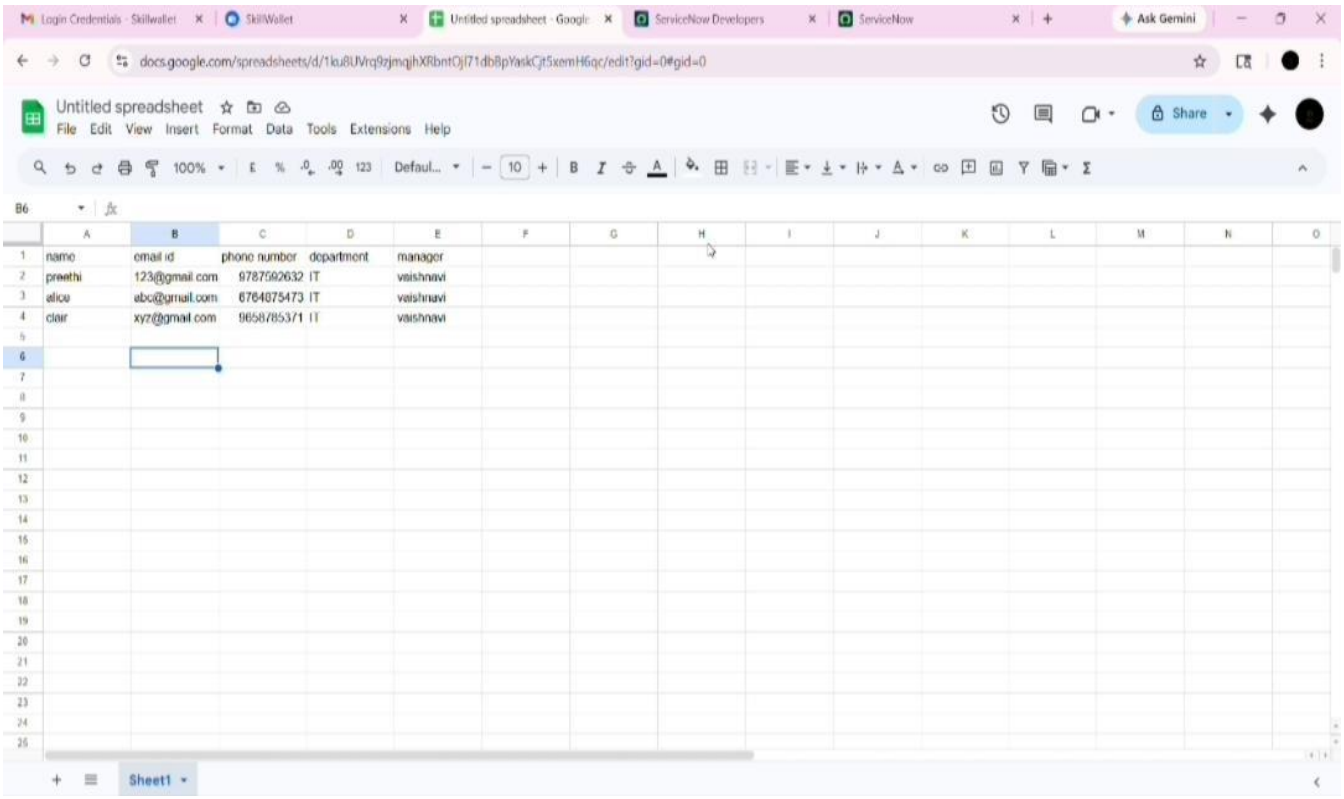
• Automate recurring data imports

• Integrate ServiceNow with external systems

Overall, importing data through a Data Source provides a structured and scalable way to move external data into ServiceNow efficiently and securely.

Create records in the Sheet

Procedure :



• Open google sheet

• Name the fields as name,email,department, manager and phone number

• Enter some records

• Save the sheet as: employee data

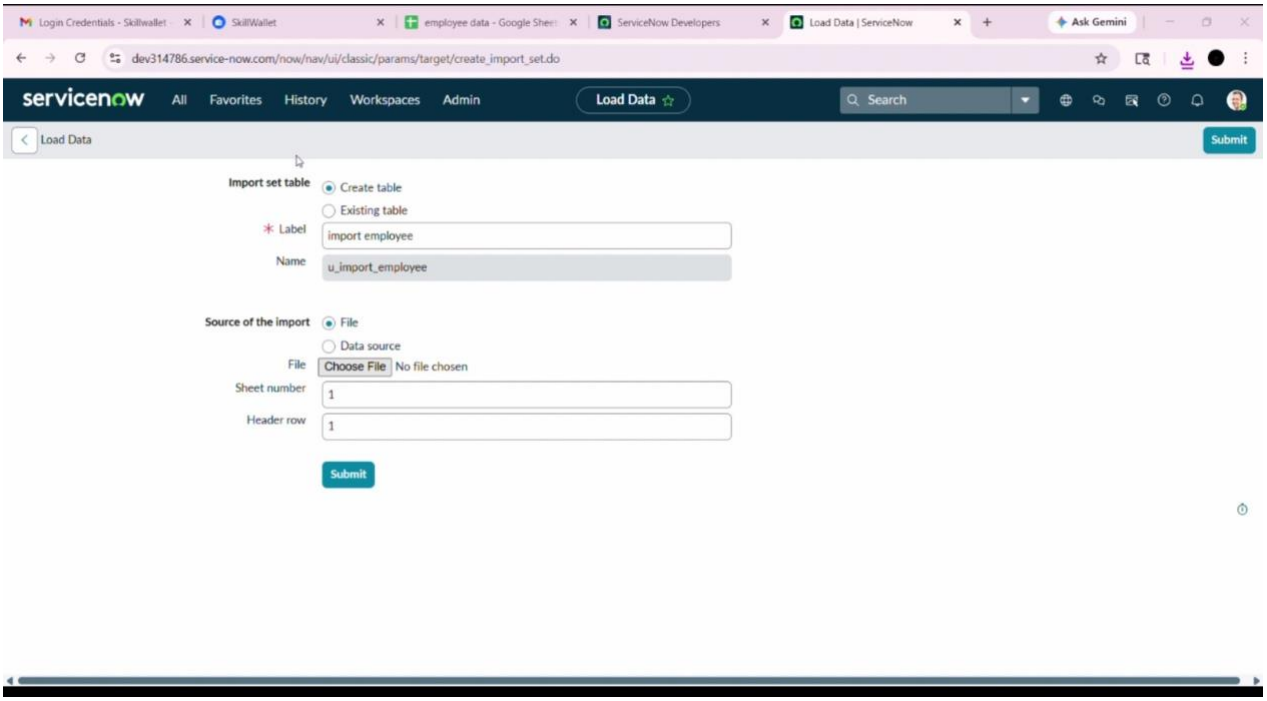
• Download the file as .xlsx , .csv (or) .xml file

Import data

The Import Data process allows employee records from an external file (CSV or XLSX) to be brought into ServiceNow using the Import Set framework. The uploaded file is first stored in a staging table (Import Set table), where the data can be reviewed and validated before being moved to the target table. This approach ensures safe data migration by separating raw imported data from production tables like sys_user. It also allows administrators to verify, transform, and control how records are inserted or updated in the system.

Creating Import Table (Staging Table)

Procedure :



• Click on All >> search for import sets

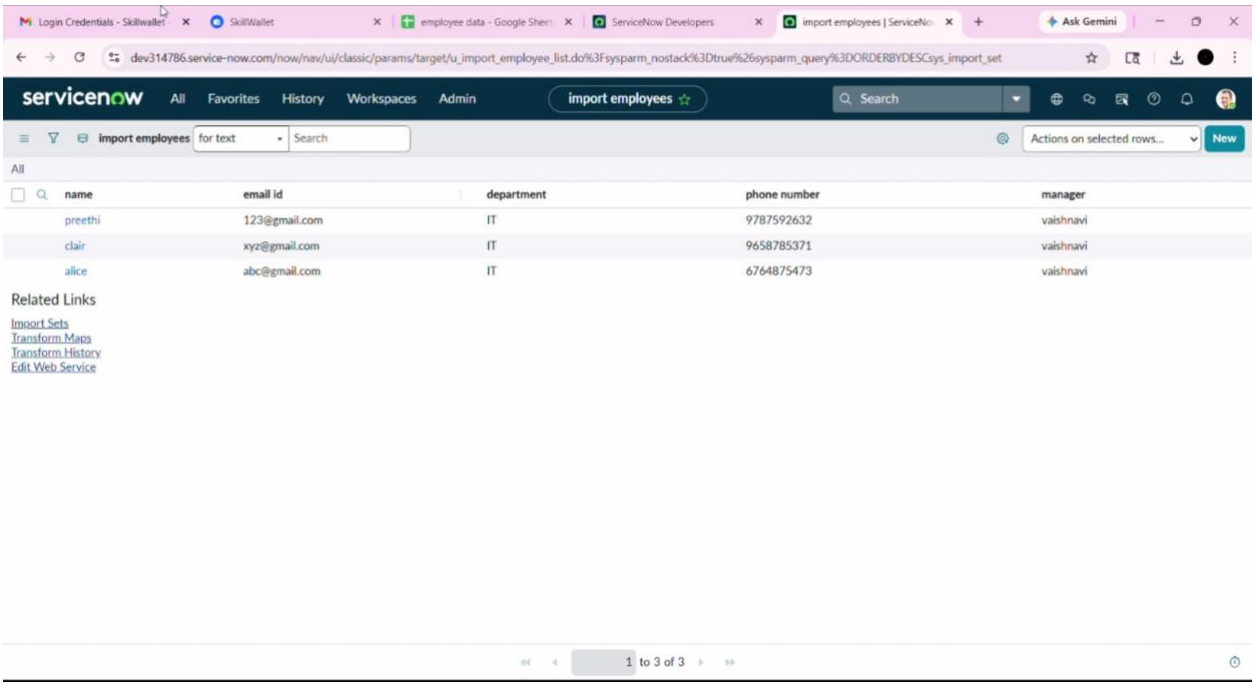
• Select load data under system import sets

• Select create table

• Give the the import set table name as: import employee

• Choose the file which we created in the XL sheet.

The data is imported,click on personalized list. Add fields to the form.



Create transform map

Procedure :

• Open service now.

• Click on All >> search for import set

• Select create transform map under import sets

• Click on new

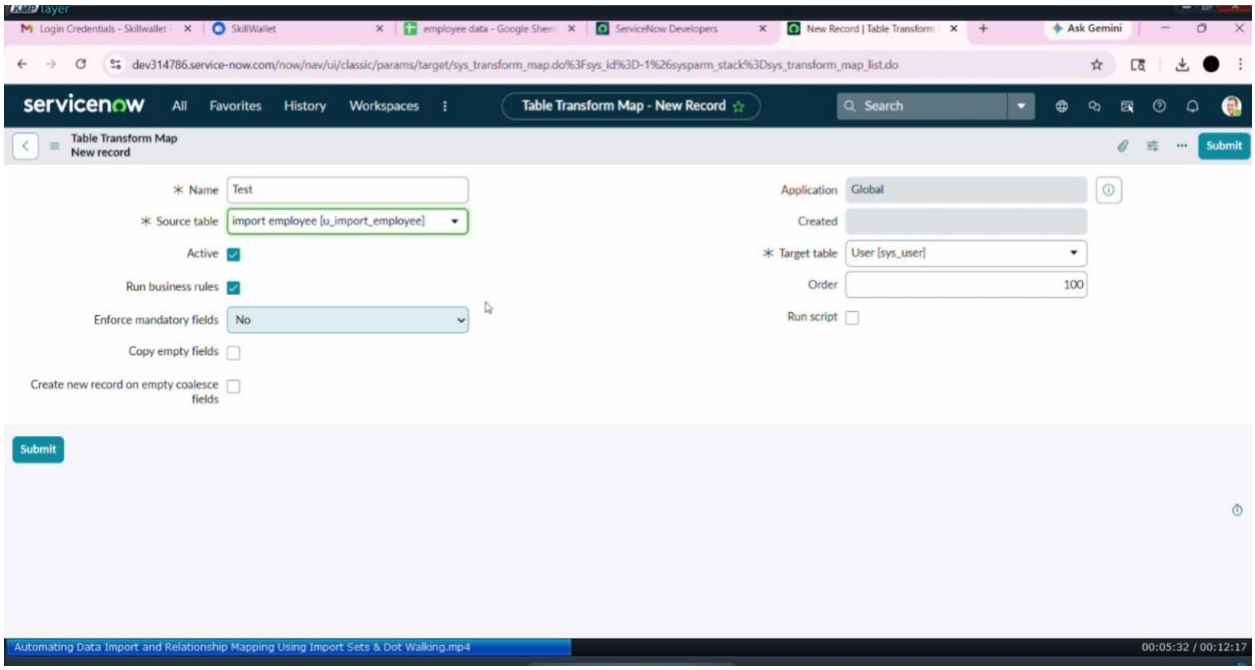
• Fill the following details to create a new transform map

• Give name as: Test

• Select source table: @123

• Select target table: user[sys_user]

• Save the form.



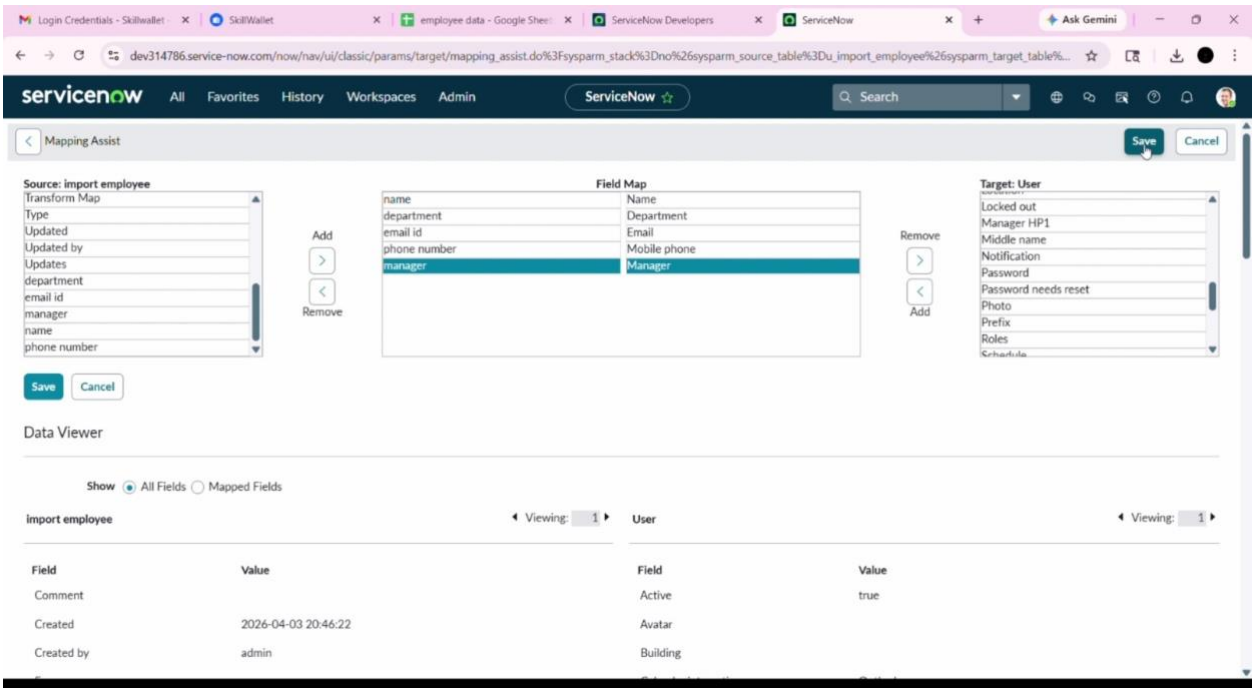
Match the fields using mapping assist

Procedure :

• Scrolled down the created transform map form

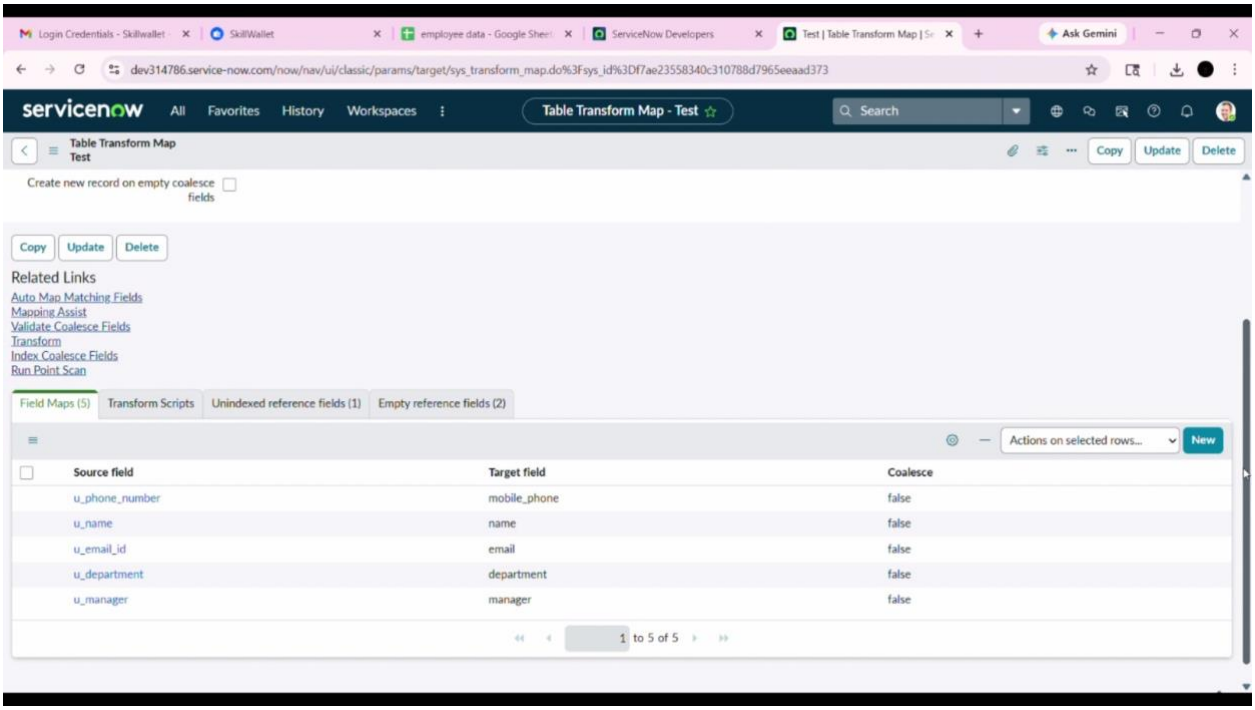
• Click on auto matching fields

• Then in the mapping fields some fields are matched



Using mapping assist map the manager field.

• Select the fields which you are mapping in the staging table and target table



• Select manager in source table and add

• Select manager field in target table

• Click on the save button.

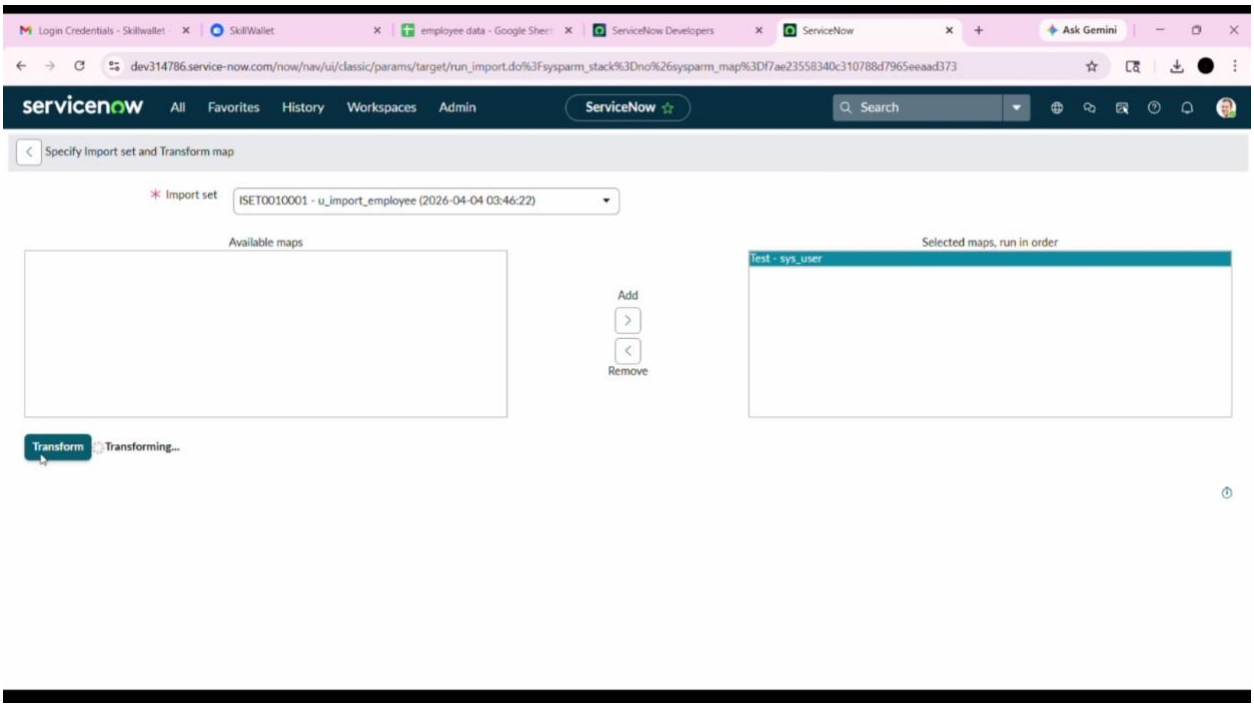
Transform the Data

Procedure :

• Click on transform in the related links.

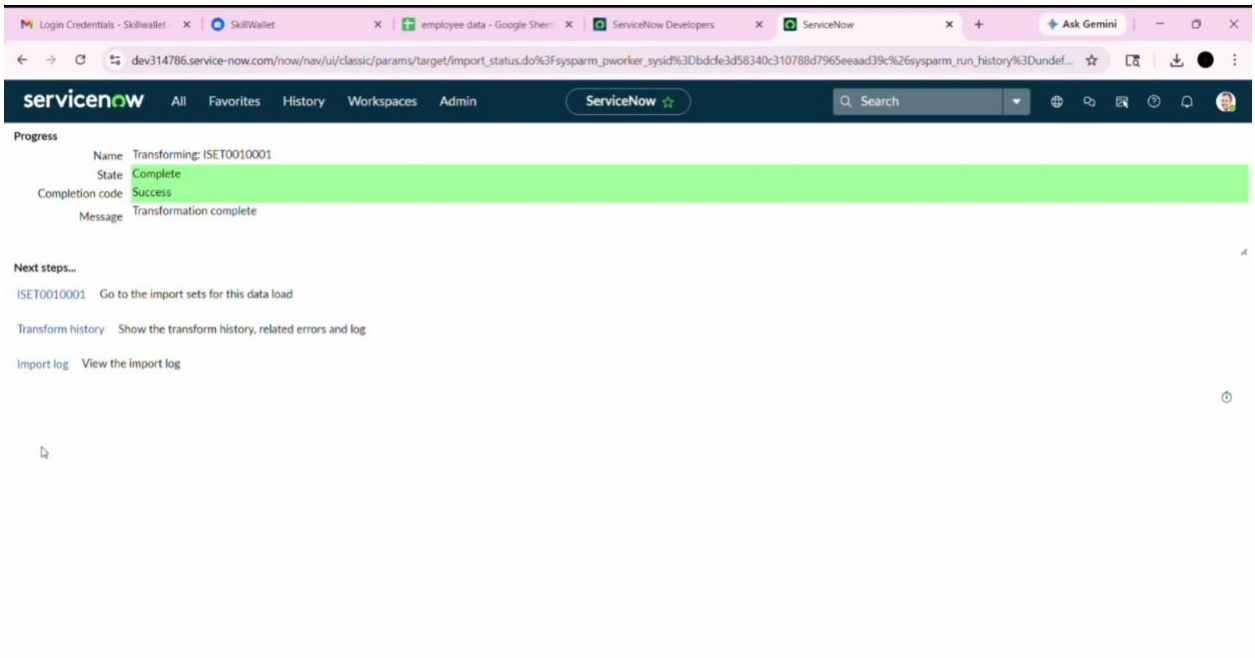
• Click on transform.

• Data's transform state is completed.



• Click on ISET0010008 under next steps

• Four rows are inserted.



Dot Walking

Dot Walking in ServiceNow is a technique used to access fields from a related table using a reference field.

When one table has a reference field pointing to another table, dot walking allows you to retrieve fields from that referenced record without writing complex queries.

Display department to the assigned to user

Procedure :

• Open service now.

• Click on All >> incident

• Select create new under incidents

• There is no field like department in the form

• Email and manager fields are not visible

• By using dot walking retrieve the data without opening the record

• Click on context menu go to configure and select form layout

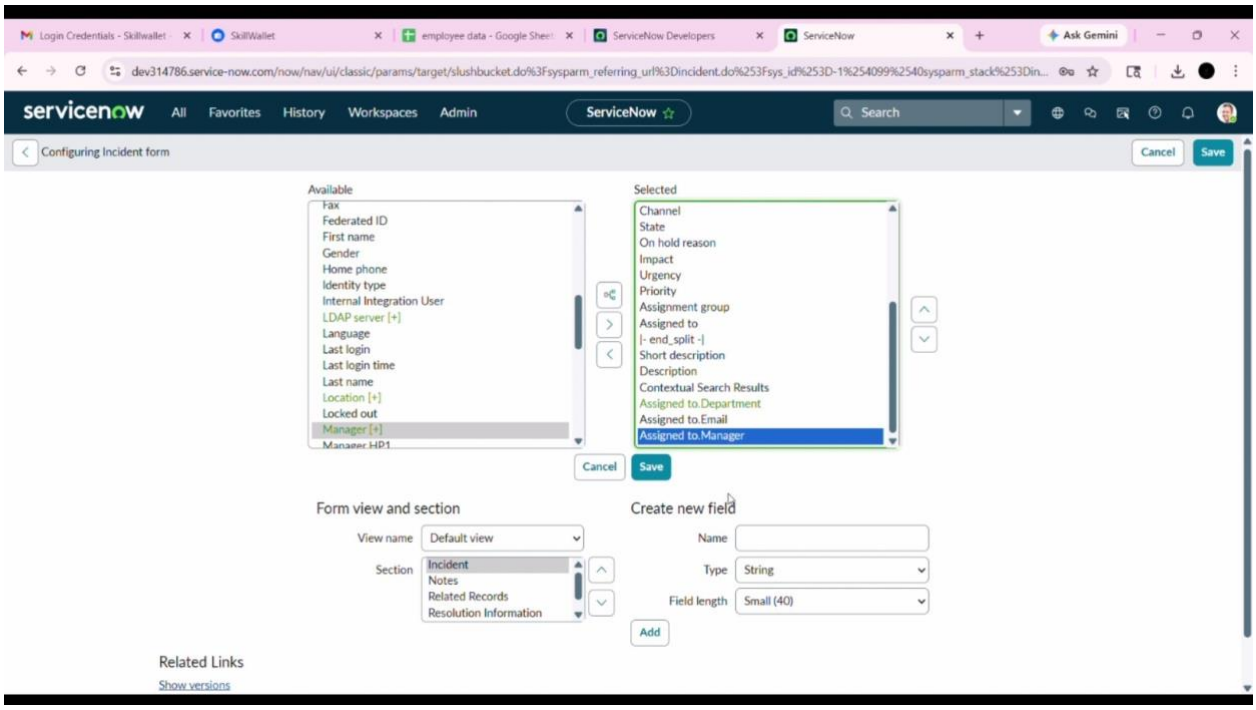
• Select assigned to.department as mentioned below

• Click on save

• Add another field assignment.email.

• Also add assignment.manager field

• Click on save.



• Open an incident form

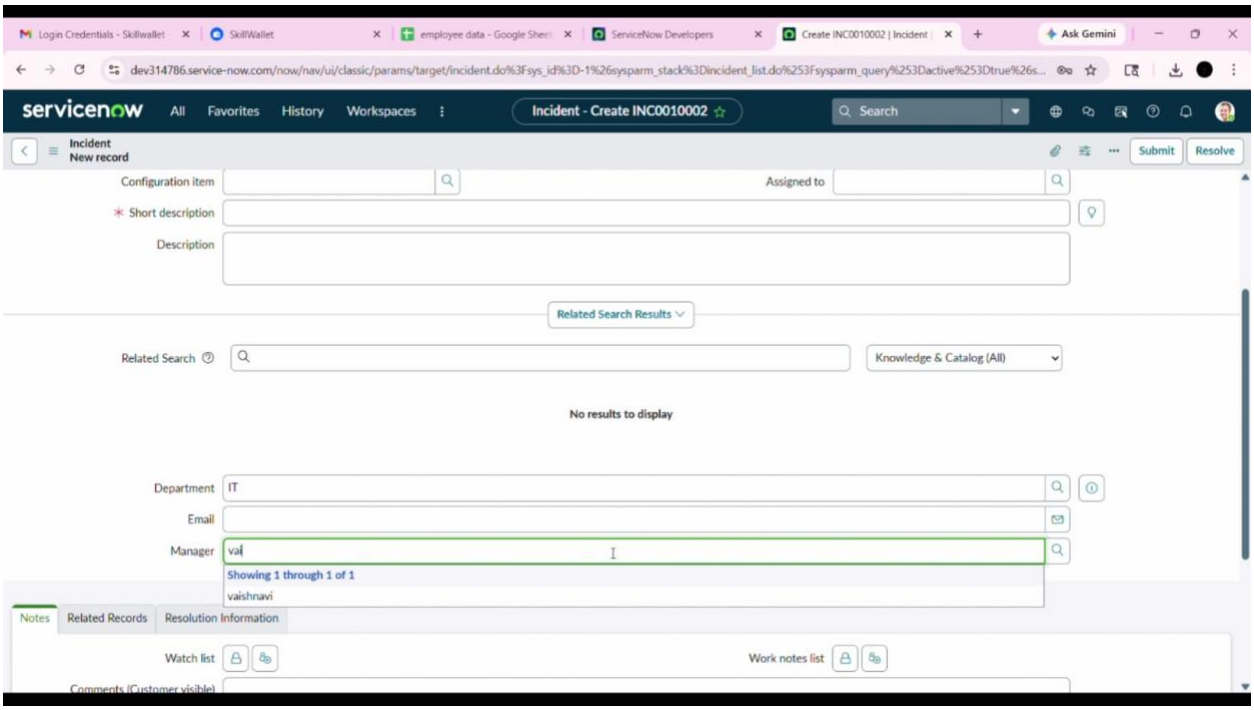
• Fill the mandatory fields and assigned to just imported user

• Then department and email field is automatically filled

Testing

Procedure :

Testing is the process of verifying that a system, feature, or functionality works as expected and meets defined requirements before it is released to users.



Conclusion

This project automates data imports using Import Sets, Transform Maps, and Dot Walking in ServiceNow PDI, enabling efficient data migration and relationship mapping.

• Import Sets allow easy data import from external sources.

• Transform Maps ensure data is correctly mapped and processed.

• Dot Walking enables automatic relationship mapping between tables.

This project demonstrates how to efficiently import external employee data into ServiceNow using Import Sets and Transform Maps while maintaining proper data relationships. By mapping reference fields such as department and manager correctly, structured and reliable records are created in the system.

Using dot walking on the Incident form further enhances usability by automatically displaying related information like department, email, and manager without manually opening the user record. Overall, the solution improves data accuracy, reduces manual effort, and ensures seamless relationship mapping across tables.

Real time examples

1. Employee Data Import (HR System)

📌 Scenario

A company uploads employee data from Excel into ServiceNow.

⚙️ Process

Excel file ⟶ Import Set

Transform Map ⟶ Employee table

Department field mapped as reference

🔗 Dot Walking Example :

Employee.department.manager.email

✅ Real Outcome

• Employee automatically linked to department

• Manager details fetched without manual entry

2. Incident Assignment Automation (ITSM)

📌 Scenario

Incidents are imported from an external ticketing system.

⚙️ Process

Import incidents via CSV

Map “Assigned Group” field

🔗 Dot Walking

Incident.assignment_group.manager.name

✅ Real Outcome

• Incident assigned to correct team

• Manager details accessible instantly

3. Asset Management System

📌 Scenario

Company imports laptop/desktop asset details.

⚙️ Process

Import asset list

Map user (owner) as reference

🔗 Dot Walking

Asset.assigned_to.department.location

✅ Real Outcome

Asset linked to employee

Department and location auto-connected

4. Customer Support (CRM Integration)

📌 Scenario

Customer data imported from CRM system.

⚙️ Process

Import customer records

Map account and contact tables

🔗 Dot Walking

Customer.account.owner.phone

✅ Real Outcome

Customer ⟶ Account ⟶ Owner relation created

• Support team gets full context instantly

5. **Student Management System (College Use Case)**

📌 Scenario

College uploads student records.

⚙️ Process

Import student Excel sheet

Map course and department

🔗 Dot Walking

Student.course.department.hod.name

✅ Real Outcome

• Student linked to course

• Department and HOD info auto available

Advantages

1. **Reduced Manual Work**

Automation removes the need for manual data entry, saving time and effort.

2. **Faster Data Processing**

Large volumes of data (Excel/CSV) can be imported quickly and efficiently.

3. **Improved Accuracy**

Reduces human errors during data entry and mapping.

4. **Better Data Consistency**

Standardized transformation ensures uniform data across tables.

5. **Easy Relationship Mapping**

Using dot walking, related data can be accessed easily (like user → department → manager).

6. **Scalability**

Can handle large datasets and repeated imports without performance issues.

7. **Reusability**

Transform maps and scripts can be reused for future imports.

8. **Integration Support**

Easily integrates with external systems (databases, APIs, files).

9. **Real-Time Data Handling**

Supports scheduled and automated imports for up-to-date data.

10. **Improved Productivity**

Developers and admins can focus on higher-level tasks instead of manual work.

SOURCE CODE :

```
<!DOCTYPE html>
<html lang=”en”>
<head>
<meta charset=”UTF-8”>
<title>Automating Data Import & Relationship
Mapping</title>

<style>
Body {
   Font-family: Arial, sans-serif;
   Background: #f4f6f9;
   Margin: 20px;
}
H1 {
   Text-align: center;
   Color: #333;
}
```

```
.card {
   Background: white;
   Padding: 20px;
   Margin: 20px auto;
   Width: 85%;
   Box-shadow: 0 0 10px rgba(0,0,0,0.1);
   Border-radius: 10px;
}
Button {
   Padding: 10px 15px;
   Background: #007bff;
   Color: white;
   Border: none;
   Border-radius: 5px;
   Cursor: pointer;
}
Button:hover {
   Background: #0056b3;
}
```

```
Table {
   Width: 100%;
   Border-collapse: collapse;
   Margin-top: 10px;
}
Th, td {
   Border: 1px solid #ccc;
   Padding: 10px;
   Text-align: center;
}
Th {
   Background: #007bff;
   Color: white;
}
</style>

</head>

<body>
```

```
<h1>Automating Data Import & Relationship Mapping</h1>

<div class="card">
  <h2>Upload CSV File</h2>
  <input type="file" id="fileInput" accept=".csv">
  <button onclick="processFile()">Import Data</button>
</div>

<div class="card">
  <h2>Import Set (Raw Data)</h2>
  <table id="rawTable"></table>
</div>

<div class="card">
  <h2>Mapped Data (Relationship Mapping)</h2>
  <table id="mappedTable"></table>
</div>
```

```
<div class=”card”>
   <h2>Dot Walking Output</h2>
   <button onclick=”dotWalk()”>Show Results</button>
   <ul id=”output”></ul>
</div>

<script>

// Import Set
Let importSet = [];

// Reference Table (Departments)
Const departments = [
   { dept_id: “D1”, dept_name: “HR” },
   { dept_id: “D2”, dept_name: “IT” },
   { dept_id: “D3”, dept_name: “Finance” }
];

// Final Table
```

```
Let employees = [];

// Read CSV File
Function processFile() {
   Const file = document.getElementById("fileInput").files[0];

   If (!file) {
      Alert("Upload CSV file first!");
      Return;
   }

   Const reader = new FileReader();

   Reader.onload = function€ {
      parseCSV(e.target.result);
   };

   Reader.readAsText(file);
}
```

```
// Parse CSV
Function parseCSV(data) {
   Const rows = data.trim().split("\n").map(r => r.split(","));
   Const headers = rows[0];

   importSet = [];

   for (let I = 1; I < rows.length; i++) {
      let obj = {};
      headers.forEach((h, index) => {
         obj[h.trim()] = rows[i][index]?.trim();
      });
      importSet.push(obj);
   }

   displayTable(importSet, "rawTable");
   transformData();
}
```

```
// Display Table
Function displayTable(data, tableId) {
   Const table = document.getElementById(tableId);
   Table.innerHTML = “”;

   If (data.length === 0) return;

   Let header = “<tr>”;
   Object.keys(data[0]).forEach(key => {
      Header += `<th>${key}</th>`;
   });
   Header += “</tr>”;
   Table.innerHTML += header;

   Data.forEach(row => {
      Let rowHTML = “<tr>”;
      Object.values(row).forEach(val => {
         rowHTML += `<td>${val}</td>`;
      });
```

```
        rowHTML += "</tr>";
        table.innerHTML += rowHTML;
    });
}

// Transform Map (Relationship Mapping)
Function transformData() {
    Employees = importSet.map(emp => {

        Let dept = departments.find(d => d.dept_id ===
emp.dept_id);

        Return {
            Emp_id: emp.emp_id,
            Name: emp.name,
            Dept_id: emp.dept_id,
            Department: dept ? dept.dept_name : "Unknown"
        };
    });
```

```
    displayTable(employees, “mappedTable”);
}

// Dot Walking
Function dotWalk() {
    Const output = document.getElementById(“output”);
    Output.innerHTML = “”;

    Employees.forEach(emp => {
        Let li = document.createElement(“li”);
        li.textContent = emp.name + “ works in “ +
emp.department;
        output.appendChild(li);
    });
}
</script>
</body>
</html>
```